Agent Responsibility Engineering: A Bounded STAMP/STPA-Complete Safety Case at the Execution Boundary

Preprint — arXiv-ready (slim).

This is the short companion to the long-form master `paper/STAMP_ARE_FULL.md`. Both papers cite the same evidence packet and source artifacts under `research/stpa/`, `research/track-b-interposition/`, `research/evidence-bundles/2026-04-26-stamp-safety-reviewer-packet-submission/`, and `formal/`.

Frozen against repository commit `653d455346e57f6a0ba37eecb4132138d923a36d` on branch `codex/are-dual-track-research`, freeze time 2026-04-26T03:54:09Z.

For the full-length manuscript (background, related work, expanded STPA, worked example, appendices), read **`paper/STAMP_ARE_FULL.md`**.

For the rewrite/handoff brief (file inventory, claim discipline, outline), read **`paper/COWORK_FULL_PAPER_BRIEF.md`**.

Abstract

Autonomous systems fail most visibly when **consequential action** escapes governance. This paper presents a **bounded STAMP/STPA-complete** safety case for **Agent Responsibility Engineering (ARE)** at a **defined execution boundary** in an open-source platform. **Completeness** is used in the STPA sense **within that boundary**: every identified hazard **in scope** is either **mitigated**, **explicitly accepted with documented residual risk**, or **bound by declared assumptions**, as recorded in the normative artifact **`research/stpa/STPA_RESOLUTION.md`**. The case is supported by **traceability** (losses, hazards, unsafe control actions, safety constraints), **component tests** (policy coprocessor, pre-execution validator, atomic execution, passport and authority services), an **interposition audit**, and **formalizable invariants** (TLA+ skeleton; not a machine-checked proof).

We **do not** claim global safety, outcome safety, policy correctness, or proof for every deployment. We **do** claim: (1) a **STAMP-aligned** control structure (controller, controlled process, feedback); (2) **causal interposition** on the golden execution path (coprocessor verdict → pre-execution validation → receipt binding → executor); (3) **fail-closed** behavior for **policy `Effect::Allow`** when the immutable **Ledger** write fails at the coprocessor (remediated critical gap: **Allow** cannot cross the API without a successful write or a downgrade to **Deny** with `deny_reason: ledger_write_failed`); (4) explicit **residual risk** and **out-of-scope** hazard classes (distributed partitions, strata composition, human escalation).

Keywords: STAMP, STPA, agent safety, execution boundary, unsafe control actions, causal interposition, governance, accountability.

1. Introduction

Large language model and agent stacks often add **guardrails** that are **observational** or **advisory**: they filter, log, or score outputs, but do not always sit on the **causal path** to **side effects**. **Causal interposition** is the property that no governed consequential action executes unless an **affirmative governance verdict** was issued **before** the execution boundary is crossed. Without it, "safety" reduces to policy theatre.

Agent Responsibility Engineering (ARE) is a platform architecture that places **policy evaluation**, **identity and authority verification**, **pre-execution validation**, **evidence-bound receipts**, and **atomic execution orchestration** on that path. This paper is not a manifesto for safe AI in the large; it is a **safety case** in the **systems** sense: **bounded**, **traceable**, and **falsifiable** against the code and tests we cite.

Contributions. (1) A **defined execution boundary** and **process model** for agent action. (2) A **STPA package** (losses, hazards, UCAs, constraints) with a **resolution record** that closes the analysis **within scope** without hiding incompleteness elsewhere. (3) **Implementation evidence** for the core path, including a **documented remediation** of a former fail-open (Allow without durable Ledger evidence at the coprocessor). (4) An explicit **claims discipline**: what is **not** claimed and where **residual risk** remains.

For background on STAMP/STPA, related work in policy enforcement and reference monitors, the formal interposition theorem with proof sketch, the worked micro-example, and the full UCA tables, see the long-form master `paper/STAMP_ARE_FULL.md`.

2. STAMP and STPA (control structure)

STAMP (Systems-Theoretic Accident Model and Processes) treats safety as **control** of a **controlled process**, with **feedback** and a **process model** the controller uses to decide when control actions (permit, deny, escalate, etc.) are appropriate [Leveson, 2012].

STPA (System-Theoretic Process Analysis) [Leveson & Thomas, 2018] builds **losses**, **hazards** (system states), **unsafe control actions** (UCAs), **causal scenarios**, and **safety constraints**, with traceability from constraints to design and verification.

We use STPA as the **structuring method** for the safety case; we do **not** claim that all of systems theory is "solved" for agents. The **controller** in our case is composed of the **policy coprocessor**, **pre-execution validator (PEV)**, and **atomic execution** engine; the **controlled process** is the agent and its **executors**; **feedback** includes the **immutable Ledger**, registries, and receipts.

3. System boundary and process model

Inside the boundary.

1. A **consequential** action is requested only through flows that reach **`s4/atomic-execution`** `execute` with a **registered** `ActionExecutor`.

2. **Execution-time** gating uses **`s1/policy-coprocessor-core`** (scope, authority, policy) and **`s4/pre-execution-validator`** (decision id, agent, passport, action class, optional admission).

3. **Evidence before side effect: `s4/action-receipt-generator`** (receipt before `executor.execute` in the engine).

Process model (conceptual; state is **distributed** across services): agent id, passport scope, delegation chain, revocation awareness, policy bundle evidence, prior verdicts, receipt and ledger evidence, and **known uncertainty** (e.g. dependency unavailability). **Invalid** models—stale, contradictory, incomplete, unverifiable—must yield **block** or **deny** at the boundary, not silent permit, **under the tests we cite**.

Out of scope for the **bounded** claim: ad hoc **Kafka** or **S2–S6** paths that do not go through the golden execution stack; **strata** behavior except as **assumptions**; **global** distributed consensus proofs; **outcome** safety (the world after a permitted action).

Governance strata (sibling repositories) may **constrain the process model** upstream; **ARE** remains **authority at execution** per project integration documentation.

4. ARE execution path (implementation sketch)

Layer	Component (repo path)	Role
Policy	`s1/policy-coprocessor-core`	Composes **Allow / Deny / Escalate / Unavailable**; **Allow** requires **Ledger** write success or **Deny** + `ledger_write_failed`
Identity	`s0/passport-issuance-engine`, `s0/agent-registry-service`, `s1/non-transferable-authority-engine`	Passports, agents, delegation
Pre-execution	`s4/pre-execution-validator`	Binds **decision** to **action** and **live** passport/agent
Execution	`s4/atomic-execution`	**Validate → receipt → execute** order
Evidence	`s4/action-receipt-generator`, `s0/immutable-ledger`	Receipts and append-only history
Edge	`sx/are-api-gateway`	**Transport**; not the semantic authority in itself

Critical invariant (coprocessor): no **`Effect::Allow`** leaves the coprocessor unless the **Ledger** write for that evaluation succeeds; otherwise the effect becomes **`Effect::Deny`** with **`deny_reason: ledger_write_failed`** and aligned evidence (`s1/policy-coprocessor-core/src/service.rs` lines 190–211; test `parallel_evaluation_and_ledger_paths`). This was a **remediated** gap (formerly **Allow** with `ledger_written == false`).

5. STPA analysis and resolution (within the defined boundary)

Losses (examples): unauthorized consequential action; PII exposure; authority stale or revoked; execution without evidence; fail-open under uncertainty; controller/ledger mismatch; upstream disagreement leaking to execution. (Full L1–L7 in `STPA_PACKAGE.md` §2.)

Hazards (H1–H7): execution without a current verdict; bad authority; revocation lag; unbound evidence; stale process model; degraded control; strata disagreement.

Resolution (normative): `STPA_RESOLUTION.md` assigns each **H1–H7** a **mitigated**, **accepted (bounded)**, or **assumption** outcome, with **residual risk** stated. For example, **H3** (revocation visibility) is **accepted** with a **bounded** operational window: PEV re-reads passport; fail-closed on read failure; no worst-case time bound in the repository. **H6** is **mitigated** for **Allow+Ledger**; **non-Allow** + failed Ledger (e.g. **Escalate**) is **accepted** and outside the **Permit** claim. Full table reproduced in `paper/STAMP_ARE_FULL.md` §13.1.

Unsafe control actions are **enumerated** by control action (Permit, Block, Modify, Revoke, Escalate, Recover) and STPA **flaw class** (e.g. provided when unsafe, not provided when required) in `UCA_ENUMERATION.md`; closure is **not** a hand-wavy list of examples.

6. Evidence

Component tests (non-exhaustive): Rust tests for `policy-coprocessor-core` (including `parallel_evaluation_and_ledger_paths`); Go tests for PEV; atomic execution `service_tests` (receipt before executor, validation failure blocks); passport and authority suites; internal gate matrix (`tools/testing/run-all-internal-gates.py`) for matrix components.

Interposition is documented in `interposition_audit.md` with governed vs. out-of-scope paths (see also the long form §14.3 for the audit table).

Python harness under `tests/stamp/`, `tests/interposition/`, `tests/adversarial/` regresses **documentation and structure**; primary proof remains **Rust/Go** tests.

Frozen reviewer packet (commit `653d455346e57f6a0ba37eecb4132138d923a36d`, branch `codex/are-dual-track-research`, freeze 2026-04-26T03:54:09Z): `research/evidence-bundles/2026-04-26-stamp-safety-reviewer-packet-submission/` including **`STPA_RESOLUTION.md`**, `pytest tests/ -q` (10 passed), and `run-all-internal-gates.py` (48 components, all pass), with `FILES.sha256` for integrity.

7. Assumptions, interposition completeness, and residual risk

Assumptions (abbreviated; full table in long form §13.3). A_fb: within the **defined** boundary, **consequential** execution produces **durable** evidence on the **governed** path (coprocessor Allow+Ledger, atomic receipt+outcome). **A_route** / **A_ex:** routing and **executor** exposure match deployment design. **A_pass** / **A_strata** / **A_dist:** infrastructure and strata as deployed.

Interposition (Option B, scoped): the **golden** path is **governed**; known **unbounded** product surfaces (ad hoc side effects, wrong gateway config) are **out of scope** of this case or require **deployment certification**, not a single proof in this monolith.

Residual risk (explicit): policy correctness; **human** escalation; **infrastructure**; **cross-region** ledger semantics; **strata** and **recovery** outside the repository. These are **not** hidden: they are **out of scope** for the **bounded** claim except where an **A_*** assumption applies.

8. Limitations and non-claims

We do **not** claim: universal fail-closed behavior in **every** subsystem; proof for **every** cluster layout; **STPA "complete"** for the **entire product** or physical world; elimination of **all** hazard classes; or replacement of **regulation** and **ops**.

Formal methods: a **TLA+** skeleton (`formal/tla/are_execution_boundary.tla`) models ordering at the execution boundary; it is a **proof obligation**, not a **TLA/TLAPS** proof.

9. Conclusion

This paper presents a **bounded STAMP/STPA-complete** safety case for **ARE** at a **defined execution boundary**. **Completeness** means: every in-scope hazard is **mitigated**, **accepted with residual risk**, or **bound by declared assumptions** in **`STPA_RESOLUTION.md`**, with **traceability**, **tests**, and an **auditable** record of a **remediated** coprocessor–Ledger path. The honest contribution is **closure discipline** in the safety **analysis** layer, aligned to **real** enforcement in code—not a claim of universal safe AI.

References

1. Leveson, N.G. *Engineering a Safer World: Systems Thinking Applied to Safety.* MIT Press, 2012.

2. Leveson, N.G. and Thomas, J.P. *STPA Handbook.* MIT, 2018.

3. Saltzer, J.H., Reed, D.P. and Clark, D.D. End-to-End Arguments in System Design. *ACM TOCS*, 2(4):277–288, 1984.

4. Schneider, F.B. Enforceable Security Policies. *ACM TISSEC*, 3(1):30–50, 2000.

5. Lamport, L. *Specifying Systems: The TLA+ Language and Tools.* Addison-Wesley, 2002.

6. NIST. *Artificial Intelligence Risk Management Framework (AI RMF 1.0).* NIST AI 100-1, 2023.

7. ARE platform artifacts: `research/stpa/STPA_PACKAGE.md`, `STPA_RESOLUTION.md`, `UCA_ENUMERATION.md`, `interposition_audit.md`, and frozen evidence bundle `2026-04-26-stamp-safety-reviewer-packet-submission`.

For the full reference list (20 entries with object-capability, GSN, XACML, TLA+ proofs, STPA-Sec, etc.) and discussion of related work, see `paper/STAMP_ARE_FULL.md` §3 and References.

Appendix A. One-sentence summary for submission metadata

This paper presents a bounded STAMP/STPA-complete safety case for ARE at the defined execution boundary. Completeness means that every identified hazard in scope is either mitigated, explicitly accepted with residual risk, or bound by declared assumptions, as recorded in STPA_RESOLUTION.md.

Version: 2026-04-26. Source file: `paper/STAMP_ARE_ARXIV_READY.md`. Long-form master: `paper/STAMP_ARE_FULL.md`.